Moreover, setting $\kappa_X = \kappa_Y = \kappa$ and taking the limit as $\kappa \to \infty$, the normalization from (27) leads to $\text{CKA}(XX^\text{T}, YY^\text{T})$:

$$\lim_{\kappa\to\infty} \frac{\sum_{i=1}^{p_1}\sum_{j=1}^{p_2} \frac{\lambda_X^i \lambda_Y^j}{(\lambda_X^i+\kappa)(\lambda_Y^j+\kappa)} \langle \mathbf{u}_X^i, \mathbf{u}_Y^j\rangle^2}{\sqrt{\sum_{i=1}^{p_1}\left(\frac{\lambda_X^i}{\lambda_X^i+\kappa}\right)^2}\sqrt{\sum_{i=1}^{p_2}\left(\frac{\lambda_Y^i}{\lambda_Y^i+\kappa}\right)^2}} \tag{34}$$

$$= \lim_{\kappa\to\infty} \frac{\sum_{i=1}^{p_1}\sum_{j=1}^{p_2} \frac{\lambda_X^i \lambda_Y^j}{\left(\frac{\lambda_X^i}{\kappa}+1\right)\left(\frac{\lambda_Y^j}{\kappa}+1\right)} \langle \mathbf{u}_X^i, \mathbf{u}_Y^j\rangle^2}{\sqrt{\sum_{i=1}^{p_1}\left(\frac{\lambda_X^i}{\frac{\lambda_X^i}{\kappa}+1}\right)^2}\sqrt{\sum_{i=1}^{p_2}\left(\frac{\lambda_Y^i}{\frac{\lambda_Y^i}{\kappa}+1}\right)^2}} \tag{35}$$

$$= \frac{\sum_{i=1}^{p_1}\sum_{j=1}^{p_2} \lambda_X^i \lambda_Y^j \langle \mathbf{u}_X^i, \mathbf{u}_Y^j\rangle^2}{\sqrt{\sum_{i=1}^{p_1}\left(\lambda_X^i\right)^2}\sqrt{\sum_{i=1}^{p_2}\left(\lambda_Y^i\right)^2}} \tag{36}$$

$$= \text{CKA}(XX^\text{T}, YY^\text{T}).$$

Overall, the hyperparameters of the canonical ridge objective make it less useful for exploratory analysis. These hyperparameters could be selected by cross-validation, but this is computationally expensive, and the resulting estimator would be biased by sample size. Moreover, our goal is not to map representations of networks to a common space, but to measure the similarity between networks. Appropriately chosen regularization will improve out-of-sample performance of the mapping, but it makes the meaning of "similarity" more ambiguous.

D.2. The Orthogonal Procrustes Problem

The orthogonal Procrustes problem consists of finding an orthogonal rotation in feature space that produces the smallest error:

$$\hat{Q} = \arg\min_Q ||Y - XQ||_\text{F}^2 \;\text{ subject to }\; Q^\text{T}Q = I. \tag{37}$$

The objective can be written as:

$$\begin{aligned} ||Y - XQ||_\text{F}^2 &= \text{tr}((Y - XQ)^\text{T}(Y - XQ)) \\ &= \text{tr}(Y^\text{T}Y) - \text{tr}(Y^\text{T}XQ) - \text{tr}(Q^\text{T}X^\text{T}Y) + \text{tr}(Q^\text{T}X^\text{T}XQ) \\ &= ||Y||_\text{F}^2 + ||X||_\text{F}^2 - 2\text{tr}(Y^\text{T}XQ). \end{aligned} \tag{38}$$

Thus, an equivalent objective is:

$$\hat{Q} = \arg\max_Q \text{tr}(Y^\text{T}XQ) \;\text{ subject to }\; Q^\text{T}Q = I. \tag{39}$$

The solution is $\hat{Q} = UV^\text{T}$ where $U\Sigma V^\text{T} = X^\text{T}Y$, the singular value decomposition. At the maximum of (39):

$$\text{tr}(Y^\text{T}X\hat{Q}) = \text{tr}(V\Sigma U^\text{T}UV^\text{T}) = \text{tr}(\Sigma) = ||X^\text{T}Y||_* = ||Y^\text{T}X||_*, \tag{40}$$

which is similar to what we call "dot product-based similarity" (Equation 1), but with the squared Frobenius norm of $Y^\text{T}X$ (the sum of the squared singular values) replaced by the nuclear norm (the sum of the singular values). The Frobenius norm of $Y^\text{T}X$ can be obtained as the solution to a similar optimization problem:

$$||Y^\text{T}X||_\text{F} = \max_W \text{tr}(Y^\text{T}XW) \;\text{ subject to }\; \text{tr}(W^\text{T}W) = 1. \tag{41}$$

In the context of neural networks, Smith et al. (2017) previously proposed using the solution to the orthogonal Procrustes problem to align word embeddings from different languages, and demonstrated that it outperformed CCA.

E. Architecture Details

All non-ResNet architectures are based on All-CNN-C (Springenberg et al., 2015), but none are architecturally identical. The Plain-10 model is very similar, but we place the final linear layer after the average pooling layer and use batch normalization because these are common choices in modern architectures. We use these models because they train in minutes on modern hardware.

Tiny-10
3×3 conv. 16-BN-ReLu $\times 2$
3×3 conv. 32 stride 2-BN-ReLu
3×3 conv. 32-BN-ReLu $\times 2$
3×3 conv. 64 stride 2-BN-ReLu
3×3 conv. 64 valid padding-BN-ReLu
1×1 conv. 64-BN-ReLu
Global average pooling
Logits

Table E.1. The Tiny-10 architecture, used in Figures 2, 8, F.3, and F.5. The average Tiny-10 model achieved 89.4% accuracy.

Plain-$(8n+2)$
3×3 conv. 96-BN-ReLu $\times(3n-1)$
3×3 conv. 96 stride 2-BN-ReLu
3×3 conv. 192-BN-ReLu $\times(3n-1)$
3×3 conv. 192 stride 2-BN-ReLu
3×3 conv. 192 BN-ReLu $\times(n-1)$
3×3 conv. 192 valid padding-BN-ReLu
1×1 conv. 192-BN-ReLu $\times n$
Global average pooling
Logits

Table E.2. The Plain-$(8n+2)$ architecture, used in Figures 3, 5, 7, F.4, F.5, F.6, and F.7. Mean accuracies: Plain-10, 93.9%; Plain-18: 94.8%; Plain-34: 93.7%; Plain-66: 91.3%

Width-n
3×3 conv. n-BN-ReLu $\times 2$
3×3 conv. n stride 2-BN-ReLu
3×3 conv. n-BN-ReLu $\times 2$
3×3 conv. n stride 2-BN-ReLu
3×3 conv. n valid padding-BN-ReLu
1×1 conv. n-BN-ReLu
Global average pooling
Logits

Table E.3. The architectures used for width experiments in Figure 6.

F. Additional Experiments

F.1. Sanity Check for Transformer Encoders

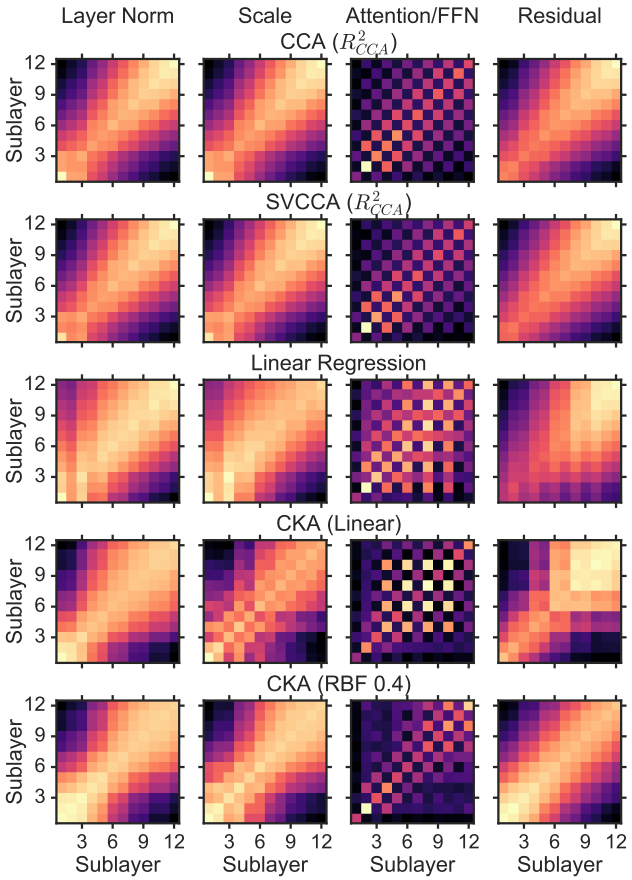

Figure F.1. All similarity indices broadly reflect the structure of Transformer encoders. Similarity indexes are computed between the 12 sublayers of Transformer encoders, for each of the 4 possible places in each sublayer that representations may be taken (see Figure F.2), averaged across 10 models trained from different random initializations.

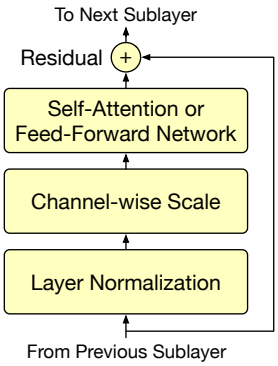

Figure F.2. Architecture of a single sublayer of the Transformer encoder used for our experiments. The full encoder includes 12 sublayers, alternating between self-attention and feed-forward network sublayers.

Index	Layer Norm	Scale	Attn/FFN	Residual
CCA ($\bar{\rho}$)	85.3	85.3	**94.9**	90.9
CCA (R^2_{CCA})	87.8	87.8	**95.3**	**95.2**
SVCCA ($\bar{\rho}$)	78.2	83.0	89.5	75.9
SVCCA (R^2_{CCA})	85.4	86.9	90.8	84.7
PWCCA	**88.5**	88.9	**96.1**	87.0
Linear Reg.	78.1	83.7	76.0	36.9
CKA (Linear)	78.6	**95.6**	86.0	73.6
CKA (RBF 0.2)	76.5	73.1	70.5	76.2
CKA (RBF 0.4)	**92.3**	**96.5**	89.1	**98.1**
CKA (RBF 0.8)	80.8	**95.8**	**93.6**	90.0

Table F.1. Accuracy of identifying corresponding sublayers based maximum similarity, for 10 architecturally identical 12-sublayer Transformer encoders at the 4 locations in each sublayer after which the representation may be taken (see Figure F.2). Results not significantly different from the best result are bold-faced ($p < 0.05$, jackknife z-test).

When applied to Transformer encoders, all similarity indexes we investigated passed the sanity check described in Section 6.1. We trained Transformer models using the `tensor2tensor` library (Vaswani et al., 2018) on the English to German translation task from the WMT18 dataset (Bojar et al., 2018) (Europarl v7, Common Crawl, and News Commentary v13 corpora) and computed representations of each of the 75,804 tokens from the 3,000 sentence `newstest2013` development set, ignoring end of sentence tokens. In Figure F.1, we show similarity between the 12 sublayers of the encoders of 10 Transformer models (45 pairs) trained from different random initializations. Each Transformer sublayer contains four operations, shown in Figure F.2; results vary based which operation the representation is taken after. Table F.1 shows the accuracy with which we identify corresponding layers between network pairs by maximal similarity.

The Transformer architecture alternates between self-attention and feed-forward network (FFN) sublayers. The checkerboard pattern in representational similarity after the self-attention/feed-forward network operation in Figure F.1 indicates that representations of attention sublayers are more similar to other attention sublayers than to FFN sublayers, and similarly, representations of FFN sublayers are more similar to other FFN than to feed-forward network layers. CKA reveals a checkerboard pattern for activations after the channel-wise scale operation (before the attention/FFN operation) that other methods do not. Because CCA is invariant to non-isotropic scaling, CCA similarities before and after channel-wise scaling are identical. Thus, CCA cannot capture this structure, even though the structure is consistent across networks.